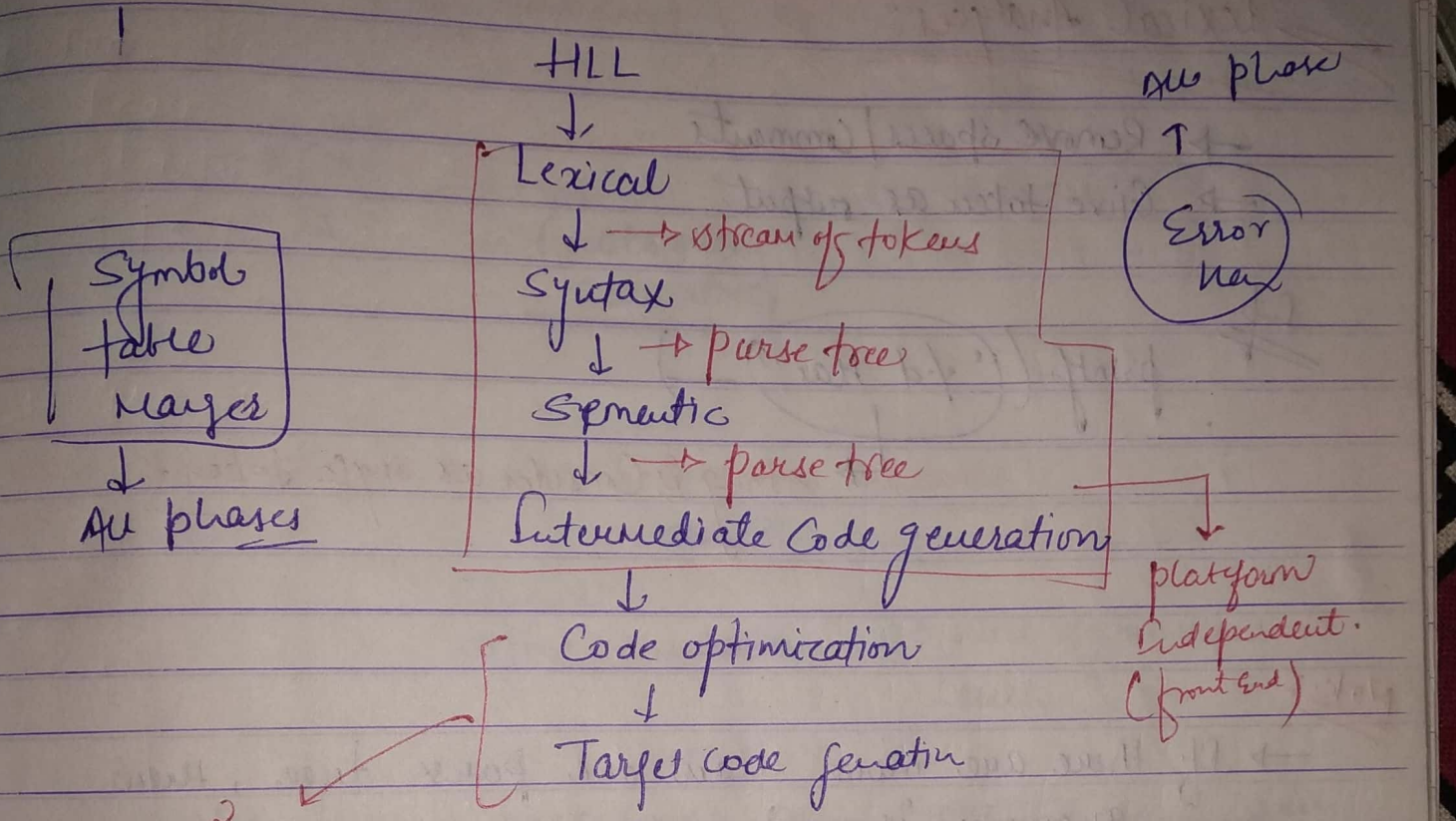
Compiler Design



Scanned by CamScanner

Lexical Analyser:

→ Remove spaces / Comments
→ Give token as output

Eg:

printf("%d Hai")

↳ String is Consider as single token.

Note:

→ If there are more than one parse tree, then grammer is ambigous.

Scanned by CamScanner

Ambigous to Un-Ambigous

eg.1 $E \to E+E$
$E \to id$
→ (make one associative)
$E \to E+id \mid id$

eg. → solve using Associativity & Precedance

It will define Recursiveness

It will define the Level.

Note: Higher precedence at ~~Level 1st then go down.~~ (+)

→ Left Associative

eg. $E \to E+T \mid T$
$T \to T*F \mid F$
$F \to id$

* is has higher precedence than + .

Right Associative

$F \to G \uparrow F$

→ Higher precedence will be at lowest Level. (+)

Scanned by CamScanner

Elimination of Left Recursion:

Left Recursive

$A \rightarrow A\alpha \mid \beta$

Right Recursive

A{
A()
α
}
A
A α
A α
β
$\beta\alpha^*$

Convert Left Recursive to Right –

$A \rightarrow \beta\alpha^*$

$A \rightarrow \beta A'$
$A' \rightarrow \alpha A' \mid \epsilon$

$A \rightarrow A\alpha \mid \beta$

eg:

$E \rightarrow E+T \mid T$
A A α β

$E \rightarrow TE'$
$E' \rightarrow +T \mid \epsilon$

Scanned by CamScanner

Rules to Eliminate Left Recursion:

(1) $A \rightarrow A\alpha \mid \beta$

$\Downarrow$

ie. $A \rightarrow \beta\alpha^*$

So,

$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' \mid \epsilon$$

(2) $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \ldots \mid \beta_1 \mid \beta_2 \mid \beta_3 \ldots$

$\Downarrow$

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \ldots$$
$$A' \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \ldots \mid \epsilon.$$

Scanned by CamScanner

(that we don't use in parsers)

Grammers

- Ambigious
- Left Recursion (due to infinite loop)
- Non-decidable

↓

At a single steps we have two or more options to go.

eg. $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \alpha\beta_3$ (+)

Backtracking might Required. → We have Common prefix.

→ to solve Non-decidable:

→ We do Left factoring.

→ eg. $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \alpha\beta_3 | \alpha\beta_4$

⇓

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 | \beta_2 | \beta_3 | \in$

Scanned by CamScanner

Left factoring:

→ procedure of converting non-decidable grammer to decidable grammer.

Imp:

↳ Decidablity can't eliminate ambiguity.

Ex:

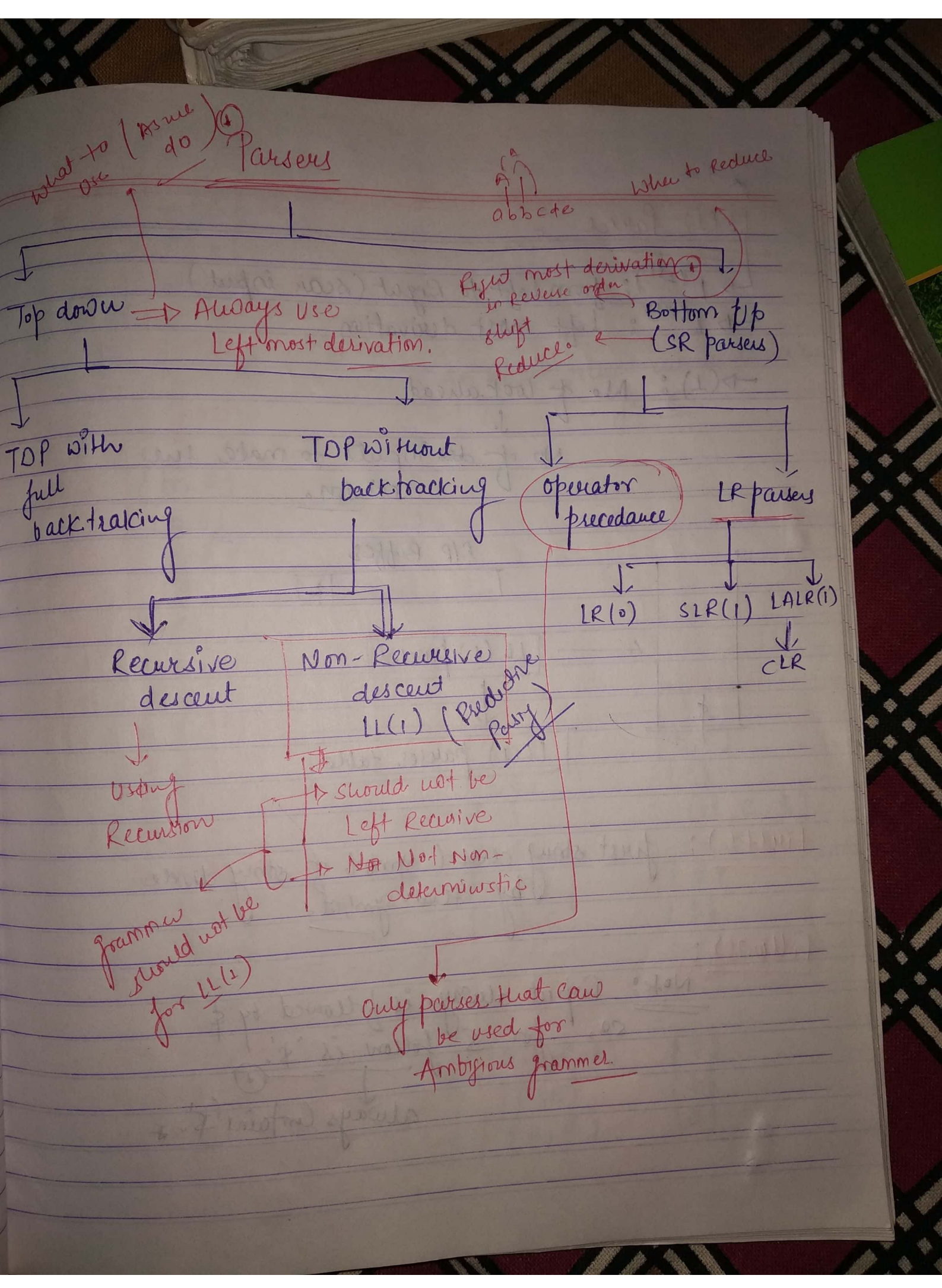
$A \to aA$
$B \to aB$

→ Not (Non-deterministic)

↓

Its deterministic (*)

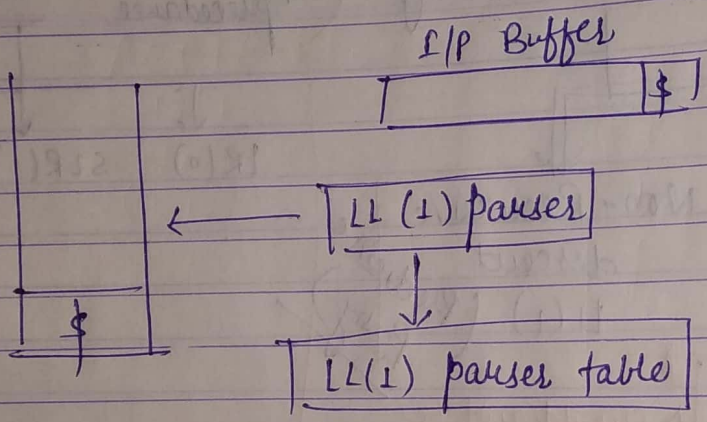
Scanned by CamScanner



Scanned by CamScanner

LL(1) Parser :

- L(1st): from Left to Right (scan input)
- L(2nd): Left most derivation
- (1): No of lookahead
  ↓
  No of ~~des~~ inputs to make the decision.



first() : first ~~string~~ ~~char~~ (symbol of string divide by that symbol.

follow() :

Note: Input always is followed by $.
so, for S follow is '$'.
↓
always Contain '$'. ✱

Scanned by CamScanner

eg. $S \to ABCD$

$A \to b \mid \epsilon$

$B \to c$

$C \to d$

$D \to e$

→ If ε in follow we will never get ε.

	first()	follow()
S	{b, c}	{$}
A	{b, ε}	{c}
B	{c}	{d}
C	{d}	{e}
D	{e}	

✓ eg.

$S \to ABCDE$

$A \to a \mid \epsilon$

$B \to b \mid \epsilon$

$C \to c$

$D \to d \mid \epsilon$

$E \to e \mid \epsilon$

	first()	follow()
A	{a, ε}	{b, c}
B	{b, ε}	{c}
C	{c}	{d, e, $}
D	{d, ε}	{e, $}
E	{e, ε}	{$}
S	{a, b, c}	{$}

✓ eg.

$S \to Bb \mid Cd$

$B \to aB \mid \epsilon$

$C \to cC \mid \epsilon$

	first()	follow()
S	{a, b, c, d}	{$}
B	{a, ε}	{b}
C	{c, ε}	{d}

Scanned by CamScanner

If ε in first(), then replace in production & check for the follow()

eg. $E \to TE'$
$E' \to +TE' / \epsilon$
$T \to FT'$
$T' \to *FT' / \epsilon$
$F \to id / (E)$

	first()	follow()
E	{id, (}	{), $ }
E'	{+, ε}	{), $ }
T	{id, (}	{+,), $ }
T'	{*, ε}	{+,), $ }
F	{id, (}	{*, +,), $ }

eg.
$S \to ACB / CbB / Ba$
$A \to da / BC$
$B \to g / \epsilon$
$C \to h / \epsilon$

	first()	follow()
S	{d, h, b, g, a}	{ $
A	{d, g, h, ε}	{h, g, $ }
B	{g, ε}	{ $, a, h, g }
C	{h, ε}	{b, h, g, $, g}

Scanned by CamScanner

LL(1) parsing ex

eg.

	first()	follow()
$E \to TE'$	{id, (}	{$,)}
$E' \to +TE' \mid \epsilon$	{+, ϵ}	{$,)}
$T \to FT'$	{id, (}	{+,), $}
$T' \to *FT' \mid \epsilon$	{*, ϵ}	{+,), $}
$F \to id \mid (E)$	{id, (}	{*, +,), $}

↓

parser table

$E' \to \epsilon$ (put in follows)

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

Note: If a cell has more than one entry then that grammar can't be generated using LL(1)

Note 1:

→ If grammer is LR & Non-deterministic that can't be generated by LL(1) parser.

→ If above is not true, still there are some grammer which can't be generated by LL(1) parser.

↓

i.e.

If a cell has more than one entry.

Scanned by CamScanner

Recursive Descent Parser:

↳ for every Variable, write fun^n. [for every Variable, one fun^n]

eg. $E \rightarrow iE'$
$E' \rightarrow +iE' | \epsilon$

```
E()
{
  if (l=='i')
    {
      match('i');
      E'();
    }
}
```

Scanned by CamScanner

▷ Video 9: (30:00)

Operator Precedence parser:

↳ Only parser that can accept ambiguous grammer.

✓ Operator grammer:

✓ $E \to E+E \mid E*E \mid id$

Note: No two Variables should be adjacent. (Imp)

↳ Should not contain ε

eg. $E \to EAE \mid id$ → Not OG.

✓ Operator Relation table:

eg. $E \to E+E \mid E*E \mid id$

	id	+	*	$
id	-	>	>	>
+	<	>	<	>
*	<	>	>	>
$	<	<	<	ε-

→ Identifier will have higher precedance compared to all other operator.

Scanned by CamScanner

→ '$' will have least precedence among all operators.

Eg. id + id * id $

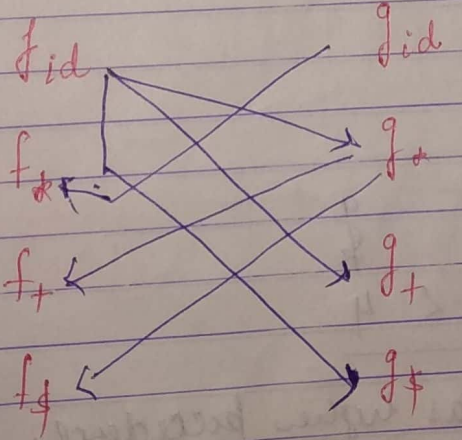
when 'top of stack' is less than input symbol then push otherwise pop. [same As Inorder to postorder]

✓ drs Advantage:

→ Size of table is $O(n^2)$.

↓

So, we will create operator fun^n table.



$(f \xrightarrow{>} g)$

$(f \xrightarrow{<} g)$

✓ same ⊕ Continue for all entry.

Scanned by CamScanner

Then

→ If there is cycle in the graph, then stop.
else

→ find the longest path for each node

eg.

$f_{id} \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$$

$g_{id} \rightarrow f_* \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$$

	id	+	*	$
f	4	2	4	0
g	5	1	3	0

⇒ size is 2n

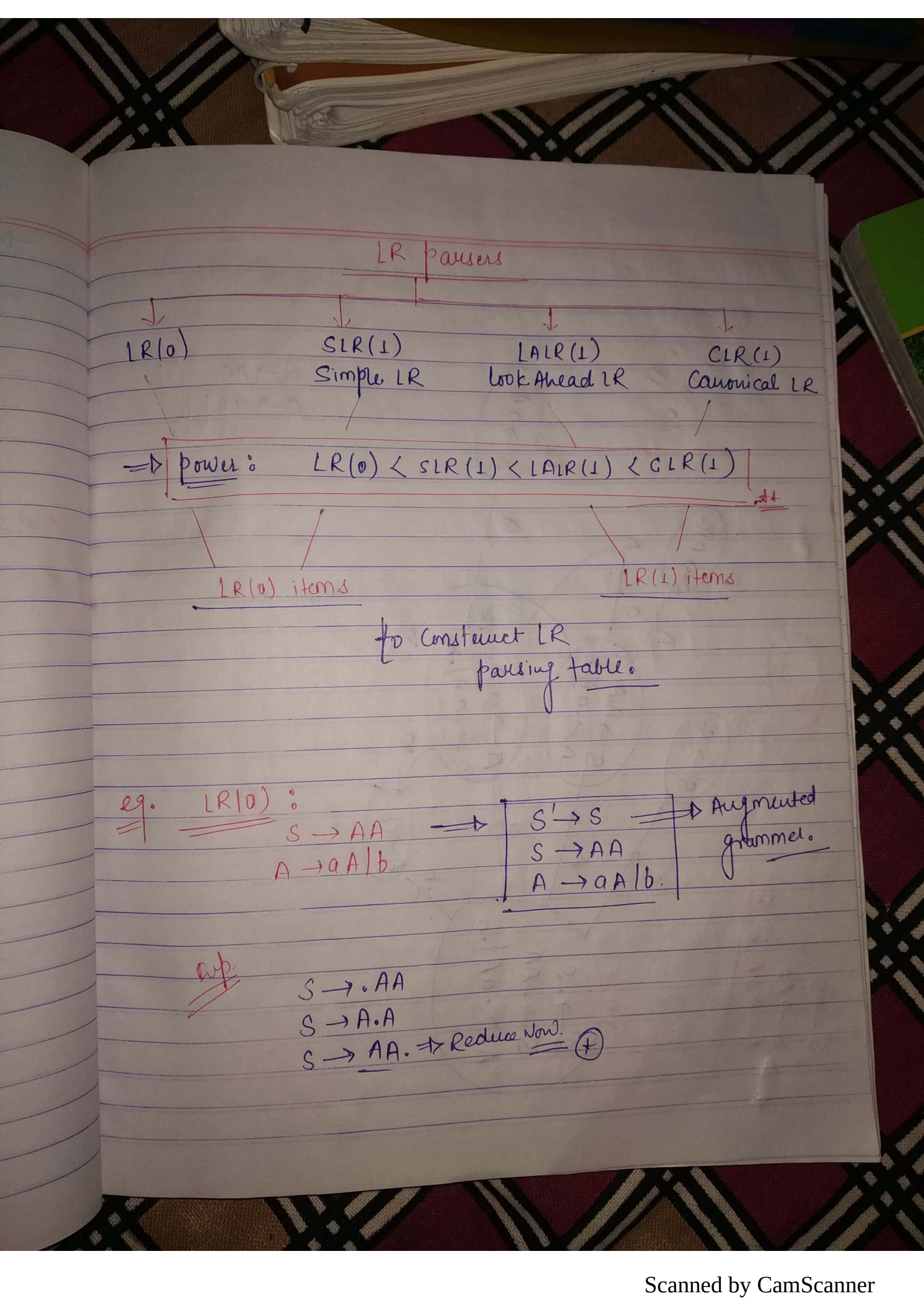
to compare

eg. f_+ g_*

2 < 4

* has higher precedence.

Scanned by CamScanner



Scanned by CamScanner

LR(0) parsing table

states	Action a	b	$	Goto A	BS
I_0	S_3	S_4		I_2	I_1
I_1			accept		
I_2	S_3	S_4		I_5	
I_3	S_3	S_4		I_6	
I_4	r_3	r_3	r_3		
I_5	r_1	r_1	r_1		
I_6	r_2	r_2	r_2		

LR(0): put Reduce(r) in all terminal for accept state.

eg. r_3, r_3, r_1 etc.

SLR(1): put reduce step only in follow of production.

eg. for state I_4 – r_3

$A \rightarrow b.$

So, check follow(A) = {$, b} ✓ ④

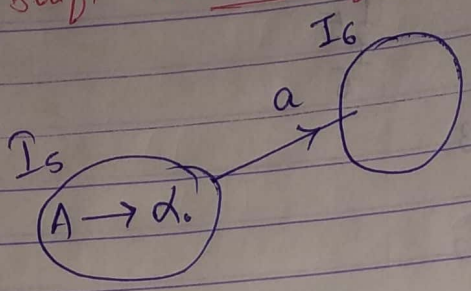
then put reduce steps only in $ & b.

not in 'a' ④

Scanned by CamScanner

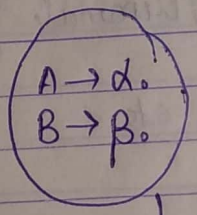
✓ LR(0) Conflicts:

① Shift - Reduce Conflict: → shift move + Reduce → Both



	a	b
I_5	S_6/r_1	r_1

② Reduce - Reduce Conflict:



	α	b
I_5	r_1/r_2	r_1/r_2

↳ If a state has two final states, then not LR(0)

✓ SLR(1) Conflict:

① Shift - Reduce: → then SR Conflict

↳ If follow(A) = {a, ...}

② Reduce - Reduce:

follow(A) ∩ follow(B) ≠ ϕ, then RR Conflict

Note

Note

Scanned by CamScanner

Note: If a grammer is LR(0), then that will also be SLR(1).

Note Eg. $S \to A|a$, $A \to a$ ⇒ Grammer is ambigious.

So,

LL(1) ✗
LR(0) ✓
SLR(1) ✓

Scanned by CamScanner

states

CLR(1) ≥ LR(0) = SLR(1) = LALR(1)

CLR(1) parser:

→ Goto & Action table will be same As LR(0) except,
the reduce moves will be only in lookaheads.

→ # states is same as # states in DFA.

Blank spaces in table means Error detecting capability.

→ CLR(1) states are more compared to LR, SLR(1) due to diff lookaheads.

LALR(1) parser:

I_3 I_6 ⇒ I_{36}
I_4 I_7 ⇒ I_{47}
I_8 I_9 ⇒ I_{89}

→ only diffn is lookaheads.

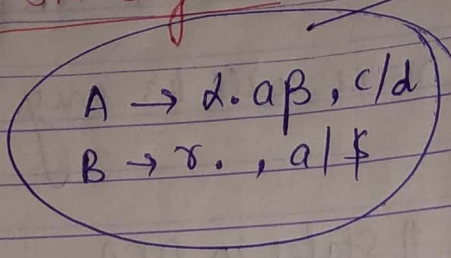
Scanned by CamScanner

Video 16

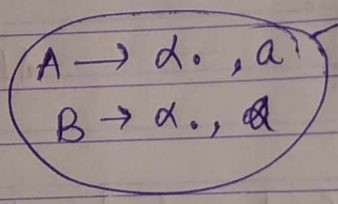
LR(1) items Conflict:

① SR Conflict:

on a,
(Shift, Reduce)



② RR Conflict:



Note: If a grammer is CLR(1) then it will ~~also be~~ not always LALR(1).

~~But Reverse is true.~~

not

If a grammer is CLR(1) then it is not LALR(1)

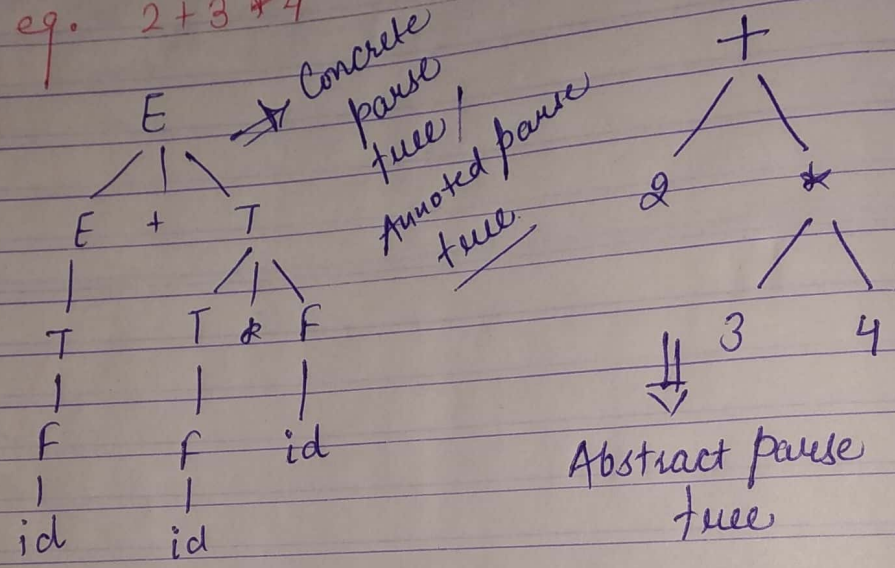
Scanned by CamScanner

Syntax Directed Translation:

Grammer + Semantic Rules = SDT

parse tree:

eg. 2+3*4



true false.

Binary to decimal:

10110.

11x2+0 = 22. ++

1111111

15x2+1 = 31

31x2+1 = 62+1 = 63 ++

for decimal print

.01 ⇒ 1/2

.11 ⇒ 3/2

.101 ⇒ 5/2

Scanned by CamScanner

(S) (L)

S-attribute SDT

- $A \rightarrow BCD$
  $A.s = f(B.s, C.s, D.s)$

Node is taking value from its children i.e. synthesized attribute.

- If only synthesized attribute
- Semantic actions only present at the end of production.

  $A \rightarrow BCC\{\}$
  [Also called post fix SDT]

- Attributes are evaluated during Bottom up.

L-attribute SDT

- $A \rightarrow BCD$
  $C.i = A.i$
  $B.i = B.i$
  $C.i = D.i$

Node is taking value from parent or sibling. (Inherited attribute).

- It have both inherited and synthesized. Each inherited attribute is restricted to take value from parent or Left side attribute only.

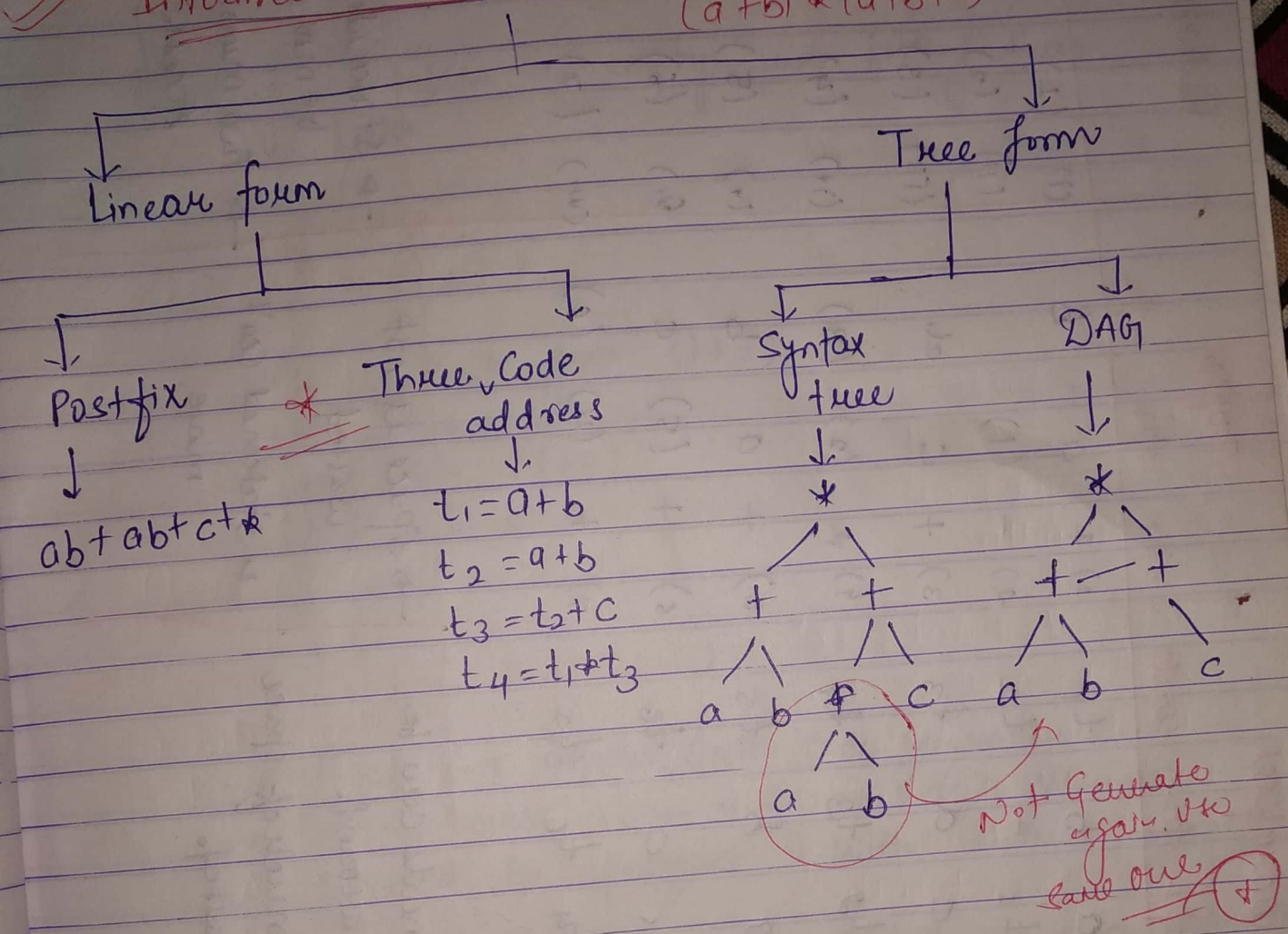
  eg. $A \rightarrow XYZ$
  $\{Y.s = A.s$ ✓, $Y.s = X.s$ ✓, $Y.s = Z.s$ ✗$\}$

- Action can be present anywhere in rhs. [$A \rightarrow \{\}BC\{\}DE\{\}F\{\}$]
- Traverse top to down, Left to Right.

Scanned by CamScanner

→ Eg. in Java we have Byte code as Intermediate code.

Intermediate Code Generation:



Types of 3-address Code:

- → x = y op z
- → x = op z
- → x = y
- → if x (rel op) y goto L
- → goto L
- → A[i] = x
  y = A[i]
- → x = *p
  y = &x.

Scanned by CamScanner

Back patching:

Leaving the tables as empty and filling them later.

eg. if (a<b) then t=1
else t=0

i) if a<b goto (i+3)
i+1) t=0
i+2) goto (i+4)
i+3) t=1

Static Single Assignment:

→ Used for intermediate code generation
→ In SSA form, each assignment to a variable should be specified with distinct names.
→ We use subscripts to distinguish each definition of variables.

GATE 2017, 16 Questions

Tip point to Remember

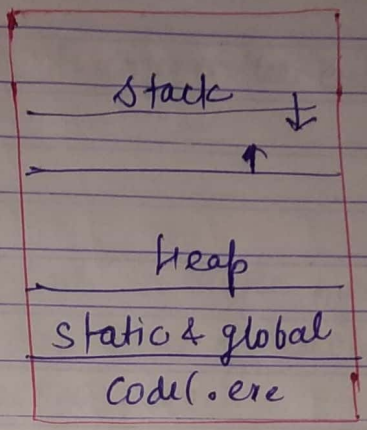
→ check whether they are asking for TEMP variables or total variables

	Quadruples					Triples			Indirect	
		opr	op1	op2	result	opr	op1	op2		
1)	$t_1 = a + b$	+	a	b	t_1	1) +	a	b	(i)	(1)
2)	$t_2 = -t_1$	-	t_1	-	t_2	2) -	(1)		(ii)	(2)
3)	$t_3 = c + d$	+	c	d	t_3	3) +	c	d	(iii)	(3)
4)	$t_4 = t_2 * t_3$	*	t_2	t_3	t_4	4) *	(2)	(3)	(iv)	(4)
5)	$t_5 = a + b$	+	a	b	t_5	5) +	a	b	(v)	(5)
6)	$t_6 = t_5 + c$	+	t_5	c	t_6	6) +	(5)	c	(vi)	(6)
7)	$t_7 = t_4 + t_6$	+	t_4	t_6	t_7	7) +	(4)	(6)	(vii)	(7)
		→ Statements can be move around. → too much of space is wasted.				→ Space is not wasted. → Statement can't move			→ Statements can be moved. → Two memory Access.	

Scanned by CamScanner

Run time Environment:

Storage Allocation Strategies:



(1) Static

- Allocation is done at Compile time.
- Binding do not change at runtime.
- One Activation record per procedure.

⇒ Variable have permanent lifetime

- Recursion is not supported.
- Size of data object at Compile time.
- Data structure can't be created dynamically.

(2) Stack:

- Recursion is possible ⇒ Nested Lifetime
- Multiple Activation Records
- Local variables will popped out with Activation ends.

(3) Heap:

⇒ Arbitrary lifetime (+)

- Allocation & deallocation can be done in any order.
- ~~first fit~~. Best fit etc.

Scanned by CamScanner

Compiler Design | Single Pass, Two Pass & Multipass

Pass : Traversal of a Compiler through the entire program.

Single Pass :
- → All phases of Compiler are in single module.
- → passes through the part of each Compilation Unit exactly once.
- → faster Compared to multipass Compiler.
- → Less efficient than multilevel pass.
- → Single pass is one that processes the i/p exactly once, so going directly from lexical analysis to Code generator & then going back for the next read.

↳ We can't optimized very well due to the Context of expressions are limited.

↳ As we can't ~~expire~~ backup & process, so grammer should be limited & specified.

↳ Generally noone use it except Pascal used it earlier not now.

Scanned by CamScanner

Two Pass / Multi Pass Compiler:

→ Compiler that process syntax tree & abstract tree of a process multiple times

↳ Two parts (first Pass – Lexical, syntax, Semantic, Intermediate Code generation

↳ second Pass (Code optimization, Code generation)

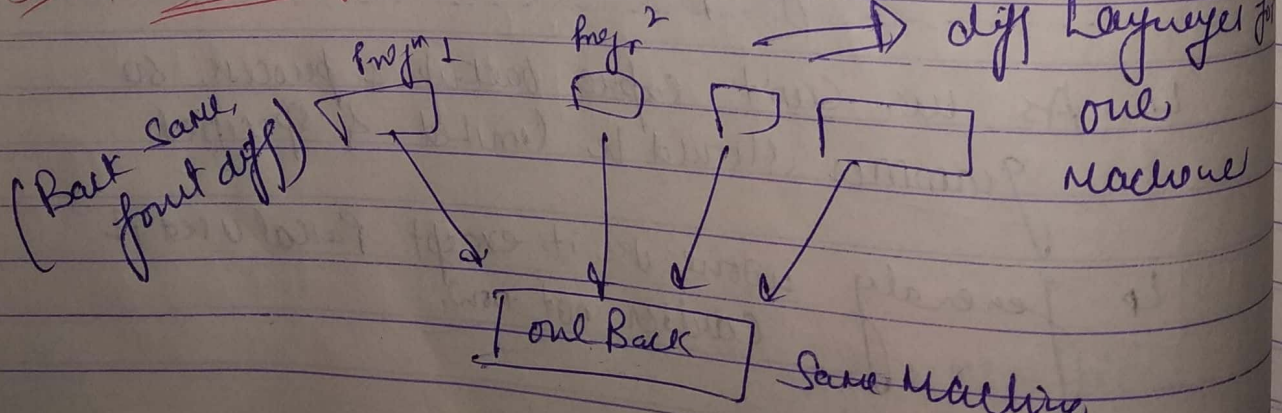
↳ first Pass:

Also called, Front end, Analytic Part, Plateform Independent.

↳ Second Pass:

Also called, Back end, Synthesis Part, Plateform dependent.

Two Advantages



Scanned by CamScanner

✓ Multiple Machine but same Programming.
(Back diff, front same)

Parameter	Single Pass	Multi
Speed	fast	slow
Memory	More	Less
Time	Less	More
Portability	NO	Yes

Scanned by CamScanner

Code optimization

Machine Independent

- Loop optimization
  [code motion or frequency reduction,
  Loop unrolling
  loop jamming
- folding - Constant propogation
- Redundancy Elimination
- strength reduction

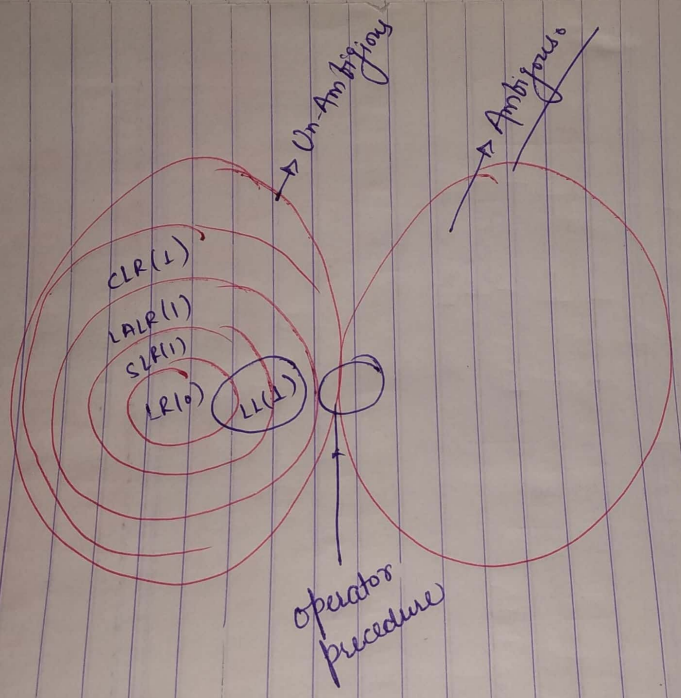
Machine Dependent

- Register allocation
- Use of addressing mode
- Peephole optimization
  - Redundant load/store
  - flow of control
  - use of m/c idioms
  - strength reduction

• key points -

✓ leader in BB.

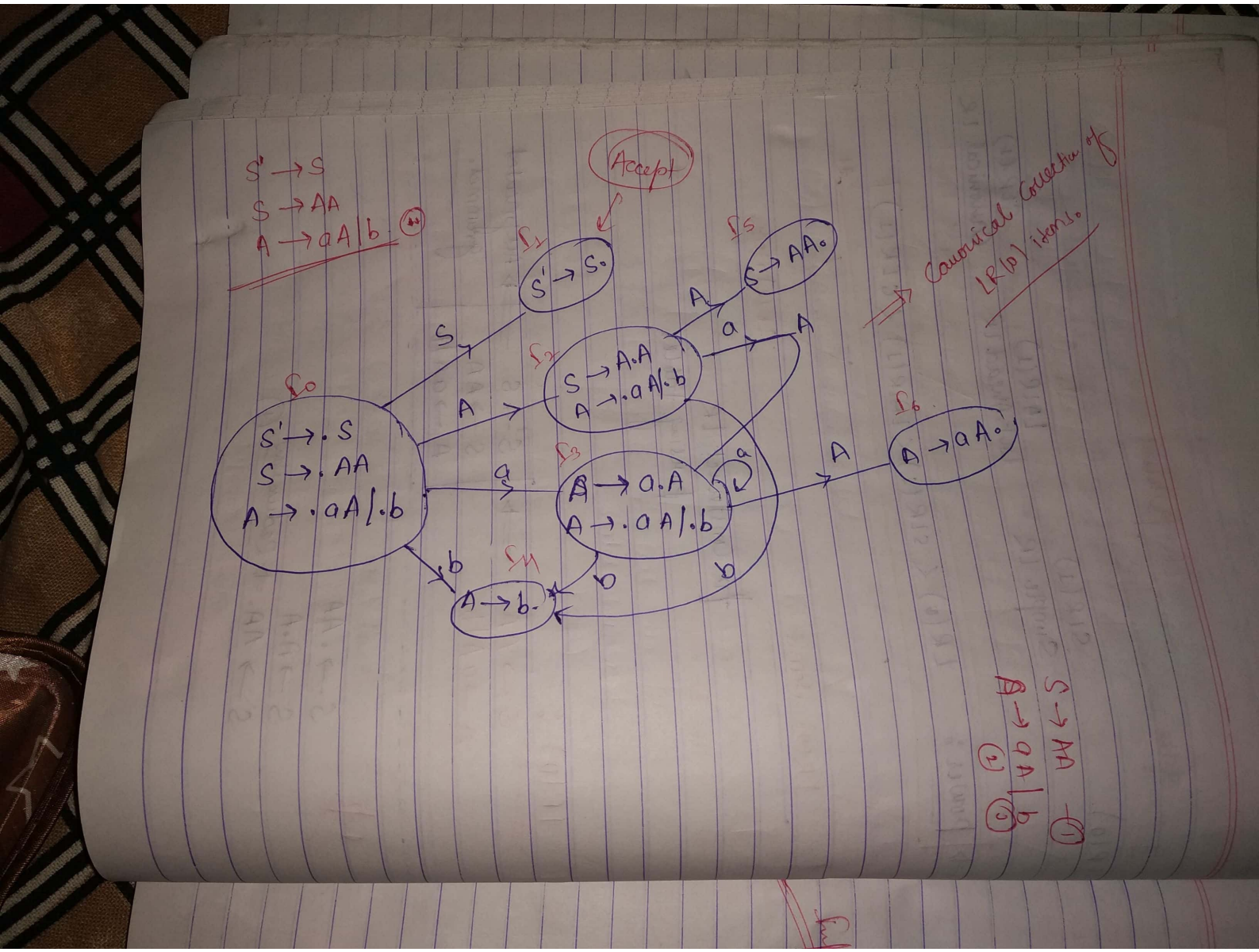
Scanned by CamScanner



Note:

→ If a grammes is LL(1) then that will definetly LALR(1) ✓✓

Scanned by CamScanner



Scanned by CamScanner